EIP Support Guide
Technical Track
Tech 4
2025/2026

Introduction

Welcome to the Technical Track of the Epitech Innovative Project (EIP) – where engineering excellence meets innovation. This track is designed for students who are passionate about pushing technological boundaries, solving complex technical challenges, and building solutions that stand out through their architectural elegance, performance, and reliability.

The Technical Track transforms you from a coder into a technical architect. You'll dive deep into emerging technologies, master the art of making informed technical decisions, and learn to build systems that are not just functional, but scalable, maintainable, and secure. This journey is about developing the technical leadership skills that distinguish great engineers from good developers.

Throughout this track, you'll embrace the full spectrum of technical excellence. You'll conduct rigorous technology evaluation and integration, architect robust and well-documented systems, engage with the open-source community, collaborate with technical experts, and optimize for performance at every level. Each objective pushes you to think critically about technology choices and their long-term implications.

This guide outlines the comprehensive framework for your Technical Track journey. It details mandatory objectives that will challenge your technical depth, as well as complementary goals you'll select with your mentor to align with your project's specific technical ambitions. Clear deliverables and evaluation criteria ensure you understand what technical excellence means in practice.

While pursuing these technical objectives, remember that you'll continue developing your project's functionality and collaborative aspects in parallel. The workload will fluctuate throughout the year, with certain periods demanding deeper technical focus. Strong time management and regular communication with your mentor and the EIP academic coordinator will be crucial. They're here to challenge your architectural decisions, validate your technical choices, and help you navigate the demanding but rewarding path of technical innovation.

Mandatory Objectives to Achieve

1 - Evaluating and Integrating New Technologies (Technology Watch)

Active technology watch

- Set up a **regular watch** on topics linked to the project: languages, frameworks, protocols, tools, standards, architectures, etc.
- Use **varied, credible sources**: technical blogs, scientific publications, official documentation, GitHub repos, newsletters, Discord, X, etc.
- Identify **emerging trends/solutions** that are **useful** for the project (not merely curiosity).

Documentation & analysis

- Write **comparative briefs** or **analysis notes** on the explored tech: advantages, drawbacks, learning curve, project fit.
- Produce **one or more technical benchmarks** between approaches/solutions (performance, scalability, maintainability, ecosystem, etc.).
- Present a **reasoned decision** on choices made (or rejected).

Concrete application

- Carry out **1–2 concrete experiments** (POC, mini-project, tested integration in the repo).
- **Integrate at least one new technology** into the project with **working proof** and justification.
- Update the project (docs, architecture, code) to reflect adopted evolutions.

Sharing & openness

- Participate actively in **tech communities**: exchanges on Discord, StackOverflow, GitHub, specialized forums, etc.
- Document your advances (**README, changelog, wiki, GitHub issues**), and share learnings within the team.

Expected deliverables (non-exhaustive)

- **Structured watch notes** or technical briefs.
- Captures/exports of bookmarks, **RSS feeds** or newsletters followed.
- **Comparative sheets** or benchmark tables.
- **POCs/mini-projects** testing technologies (separate repos or dedicated branches).
- **Changelog/README** detailing the integration performed.
- **Documented rationale** for final (or rejected) choices.
- **Community contributions** (links/captures to GitHub, forums, Discord, etc.).
- **Impact synthesis** on the project (performance, security, quality, dev time, …).

What is not sufficient

- Reading a few articles with **no testing** or application.
- Integrating a tech with **no documentation** or critical analysis.
- Choosing a solution **only because it's trendy**.
- Reusing a tech already mastered by the team with **no new exploration**.
- Producing **nothing concrete** from the watch.

2 - Structure, document, and harden the project's technical architecture

Clear, justified architecture

- Present the project's architecture: **components, services, dependencies, data flows, deployment, versioning**, etc.
- **Justify** structural choices: selected technologies, code organization, databases, APIs, frameworks, etc.

Complete technical documentation

- Provide a **structured README** including:
  - project purpose,
  - install/deploy,
  - folder/module structure,
  - key commands,
  - environment variables,
  - technical prerequisites.
- Create **at least one advanced document**: architecture diagram, technical use case, business logic, **technical roadmap**, etc.

Quality, reliability & security

- Enforce **code standards** (linting, conventions, naming, typing).
- Run **quality analysis tools** (SonarQube, ESLint, Pylint, etc.).
- Provide **unit or validation tests** on **at least one critical module**.
- Implement initial **error handling, logging, and minimal security** (auth, sanitization, backups, …).

Expected deliverables (non-exhaustive)

- **Complete README**.
- **Architecture/deployment diagram**.
- **Changelog** or **technical plan**.
- Example **config files** (dev/prod).
- **Code-quality reports** (ESLint, Sonar, etc.).
- Screens/results of **unit/stress/validation tests**.
- **Technical wiki/Notion** or annex documentation.
- **Install/deploy scripts** (Dockerfile, bash script, etc.).

What is not sufficient

- A project with **no README** or an empty one.
- A **monolithic**, non-modular code structure.
- **No** error-handling or configuration management.
- An unjustified stack ("because we know React").
- No documentation **beyond the code** itself.

Complementary Objectives (2 to be chosen by the group, in consultation with the mentor)

A - Protect and valorize your technology

Legal protection

- Identify project elements to **protect**: algorithm, method, structure, architecture model, proprietary data, etc.
- File a **protection document** (e.g., Soleau envelope, prior-art proof, timestamped digital deposit).
- Use **confidentiality clauses/agreements** (NDA, collaboration charter) when needed (open innovation, partners, external testers…).

Technical protection

- Implement **security measures** suited to your stack: encryption, anonymization, access limitation, segregated architecture, logs…
- Run a **security review**: internal audit, static analysis tool, light pentest.
- **Document** your protection strategy: what is protected, **why**, and **how**.

External valorization

- Clearly present your project's **unique technical value** (architecture, performance, innovation, robustness…).
- Draft a **tech brief/page** for partners or recruiters.
- Identify **valorization levers**: publication, **thoughtful open-source** release, controlled diffusion, secure demo.

Expected deliverables (non-exhaustive)

- **Proof of deposit** (Soleau, digital vault, registered letter, etc.).
- **Signed NDA** or template used.
- **Technical protection plan** or **map of sensitive areas**.
- **Security report** (tests, audit, automated tools…).
- **Tech fiche / architecture** highlight.
- **Diffusion/publication strategy** description (with rationale).
- README or site presenting the **valorized part** without exposing protected elements.

What is not sufficient

- Saying “we won’t communicate about the tech” **without justification**.
- Keeping the code private **without protection** or strategy.
- Using an **NDA with no context** or external counterparty.
- Failing to **document** protection or security choices.
- Claiming “technical innovation” with **no proof** of differentiation/rigor.

B - Grow community contributions

License choice & clarity

- Choose an **open-source license** (MIT, GPL, Apache, etc.) with justification.
- Add a clear **LICENSE** file and a **README** that explains permitted uses.
- Explain contribution boundaries (what's open, what isn't, under which conditions).

Call to the technical community

- Identify **relevant communities** (Discords, GitHub Topics, Reddit, specialized forums, Matrix servers, etc.).
- Create **1–2 clear calls** for contribution: open issue, public discussion, project presentation post.
- Be able to **receive and integrate at least one external contribution** (even small).

Contribution documentation

- Provide clear docs: **CONTRIBUTING.md**, repo structure, issue labels, install/repro instructions.
- Track exchanges/feedback/suggestions (**issues, PRs, discussions, comments**).
- Highlight contributors: citation, mention, recognized role.

Expected deliverables (non-exhaustive)

- Public repo with **README, LICENSE, CONTRIBUTING.md**.
- **Exchange history** (opened issues, GitHub discussions, forum threads…).
- Screens/logs of exchanges in communities (Discord, Reddit, Slack…).
- **Proof of contribution received**: pull request, comment, improvement, fork.
- **Open-source badge** (shields.io), topics, stars.
- **Testimonial** from an external user/contributor.

What is not sufficient

- Public GitHub repo **without README, license, or docs**.
- Publishing **no call for contribution** and not engaging any community.
- A project too **monolithic/opaque** to be contributive.
- Refusing feedback with "we decided so".
- Ignoring proposed contributions or **failing to track** them.

C - Collaborate with technical experts

Identify & approach experts

- Clearly define the **technical need** or topic to challenge: architecture choice, framework, security, performance, scalability, AI, etc.
- Proactively search for **relevant experts**: GitHub contributors, LinkedIn engineers, community leads, teachers, CTOs, alumni, etc.
- **Professional outreach**: clear message, precise question, appropriate channel.

Structured collaboration

- Run a **documented exchange**: meeting, written feedback, Discord/GitHub thread, code review, etc.
- Optionally organize a **review session** with an external expert (pair programming, code review, light audit).
- Implement **concrete actions** from the exchange: structure adjustment, technical plan change, new resources, tool adoption, process improvement.

Capitalize on the exchange

- Write a **summary/memo**: discussed points, advice received, decisions taken.
- Add the collaboration to the **roadmap** or documentation.
- Valorize when possible: testimonial, mention, quote, public thanks.

Expected deliverables (non-exhaustive)

- **Expert sheet**: name, profile, link, role.
- **Outreach message** (copy/capture/link).
- **Proof of exchange** (meeting, Slack, message, GitHub comment…).
- **Memo** or post-collaboration analysis note.
- Capture/list of **changes implemented** after the exchange.
- **Expert testimonial** (optional but valuable).

What is not sufficient

- Sending a **vague message** ("can you look at our project?") without a clear ask.
- Citing a verbal/informal chat **without trace** or application.
- Repeating only what the **EIP mentor** said without seeking an external view.
- **No notes** taken, **no change** made after the exchange.
- Contacting only **close network** (classmates, former teammates…).

D - Measure, test, and optimize technical performance

Define key technical metrics

- Choose **2–3 performance indicators** relevant to the project (latency, response time, memory usage, load time, bundle size, CPU, API calls, etc.).
- Integrate these metrics into the **dev cycle** (dashboard, logs, third-party tools, tests, etc.).

Set up automated/manual tests

- Run **load tests**, **stress tests**, and/or simulation scenarios.
- Run **resilience tests**: error handling, interruptions, unexpected behavior.
- Run **comparative efficiency tests**: optimized vs. initial version.

Implement optimizations

- Analyze **bottlenecks** and friction points.
- Modify one or more technical bricks (queries, processing, cache, code, …).
- **Justify trade-offs** (quality vs. performance vs. cost vs. time).

Expected deliverables (non-exhaustive)

- **Chosen indicators** + tracking table.
- **Before/after** test results (logs, charts, captures).
- **Scripts/tools** used for tests (e.g., JMeter .jmx, Docker, etc.).
- **Monitoring/benchmark dashboards** (screenshots).
- **Analysis notes** + applied optimization plan.
- **Technical rationale** for improvements or encountered limits.

What is not sufficient

- Claiming "**it runs fast on our machines**".
- **Blind optimization** with no metrics or tests.
- Relying only on **intuition/habit**.
- Only reducing image/static file size **without measured impact**.
- Failing to **document** obtained results.